

Reviewer Pack — Minimal Rank of Quandle Representations vs Boolean Function ...

Entry 6a64f508c979 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 18:06:45 UTC

Minimal Rank of Quandle Representations vs Boolean Function Permutation Complexity

Entry ID: 6a64f508c979 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 18:06:45 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Boolean Function Permutation Complexity

Statement:

[‘For every boolean function f with n variables, there exists a quandle representation Q such that the minimal rank of the associated quandle is lower bounded by $2^n / \theta(n)$, where $\theta(n)$ is the inverse Ackermann function.’, ‘This bound holds for all instances with $n \leq 40$ and is independent of the specific boolean function.’, ‘A single counterexample, where a quandle representation with a minimal rank less than $2^n / \theta(n)$ exists, would falsify this conjecture.’]

Rationale (proposer’s reasoning):

[‘Quandles are algebraic structures that have been studied in abstract algebra and combinatorics. Their representations can be used to encode information about boolean functions.’, ‘Boolean function permutation complexity measures the minimum number of permutations required to sort a list of values according to the values of the function. This is related to communication complexity and other computational tasks.’, ‘Linking quandle theory with boolean function permutation complexity may reveal new insights into both fields, potentially leading to lower bounds on complexity measures.’]

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0555f708586e498d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if no quandle representation with a minimal rank less than $2^n / \theta(n)$ exists for all n in $\{5, \dots, 40\}$ and any boolean function f , across 30 different random seeds. It is falsified if at least one seed produces a quandle representation with a minimal rank below the lower bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math

def inverse_ackermann(n):
    if n == 0:
        return 1
    elif n == 1:
        return 2
    else:
        a = 2
        for _ in range(1, n):
            a = 2 * inverse_ackermann(a - 1)
        return a

def boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def generate_quandle_representations(boolean_func):
    # Simplified representation generation
    return [boolean_func]

def calculate_minimal_rank(quandle_representation):
    # Minimal rank calculation (simplified)
    return len(quandle_representation)

def check_conjecture(n, boolean_function):
    quandle_reps = generate_quandle_representations(boolean_function)
    for rep in quandle_reps:
        minimal_rank = calculate_minimal_rank(rep)
        lower_bound = Fraction(2**n, inverse_ackermann(n))
        if minimal_rank < lower_bound:
            return False
```

```

    return True

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0.0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        boolean_func = boolean_function(n)
        if not check_conjecture(n, boolean_func):
            conjecture_holds = False
            counterexample = f"n={n}, minimal_rank<{2**n / inverse_ackermann(n)}"
            break

    return {
        "metric_name": "minimal_rank",
        "metric_value": total_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        # Default list of 30 primes
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, min(30, len(primes)))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = (sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))**0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc7c6a56.py", line 84, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc7c6a56.py", line 59, in run_trial
    if not check_conjecture(n, boolean_func):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc7c6a56.py", line 44, in check_conjecture
    lower_bound = Fraction(2*n, inverse_ackermann(n))
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc7c6a56.py", line 26, in inverse_ackermann
    a = 2 * inverse_ackermann(a - 1)
           ^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc7c6a56.py", line 26, in inverse_ackermann
    a = 2 * inverse_ackermann(a - 1)
           ^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bc7c6a56.py", line 26, in inverse_ackermann
    a = 2 * inverse_ackermann(a - 1)
           ^^^^^^^^^^^^^^^^^^^^^^^
```

[Previous line repeated 994 more times]

RecursionError: maximum recursion depth exceeded

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a maximum recursion depth exceeded error, which prevented it from producing data. | next: Investigate the cause of the recursion error in the inverse_ackermann function and ensure that the test can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11750
2	preregistration	ollama_remote	glm4:latest	0	0	6238
3	novelty	ollama_remote	glm4:latest	0	0	4768
4	novelty	ollama_remote	glm4:latest	0	0	5310
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39858
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9437
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8607
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10263
9	judge	ollama_remote	glm4:latest	0	0	29392

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
---	-------	----------	-------	-----------	-----	--------------

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125623 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/6a64f508c979.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6a64f508c979.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6a64f508c979.tar.gz (if generated)